UNIVERSITY OF Southampton
School of Electronics and Computer Science

Comp1202: Introduction III

Java

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Main Course Structure

- Two Lectures Each Week
  - Monday 13:00 – 32/1015
  - Tuesday 16:00 – 32/1015

- Also a Lab each week (worth **20%**)
  - Compulsory (must sign for attendance each week)
  - Wednesday 09:00 or 11:00
  - ECS Labs (Bld 59, Zepler, Level 2)
  - You have been allocated via timetabling
  - 10 labs, marked in weeks 4/7/10

- Other Assessment
  - Coursework (set in week 4, due in week 10, worth **40%**)
  - Multiple Choice Exam (after X-mas, worth **40%**)

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Additional Streams

- Space Cadets
  - For people who are more experienced programmers
  - Run by Son
  - Optional weekly challenge and discussion
    - Thursday 11:00 in 46/2005
    - Weeks 1-10 (i.e. starts this week!)

- Ground Controllers
  - For people who are new to programming
  - Run by Son and Jian
  - Optional weekly workshop
    - Tuesday 10:00 – 44/1061 (overflow 46/310
    - Weeks 2-10 (i.e. starts next week!)

Content

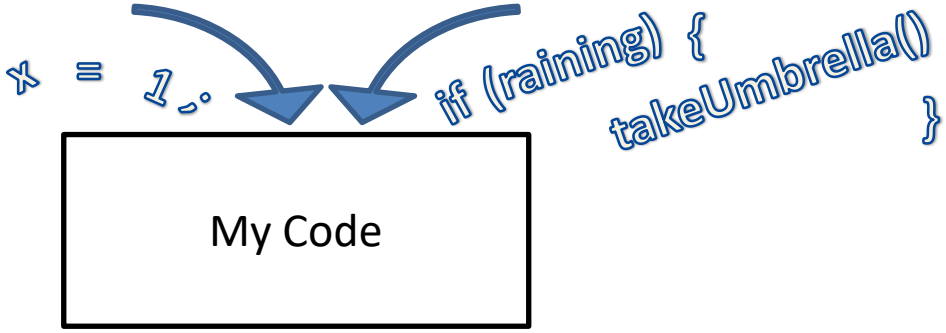
- How Java Works: The JVM
- Writing a Class in Java
  - Class
  - Member Variables
  - Method
  - Statement

- Magic incantations
  - The main() method
- A First Example
  - Defining an Account class
  - If and Boolean operations
- Introducing Toolbox

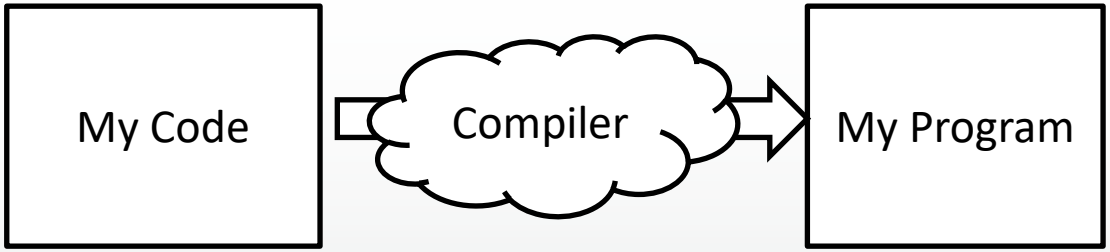
How Java Works

From code to program

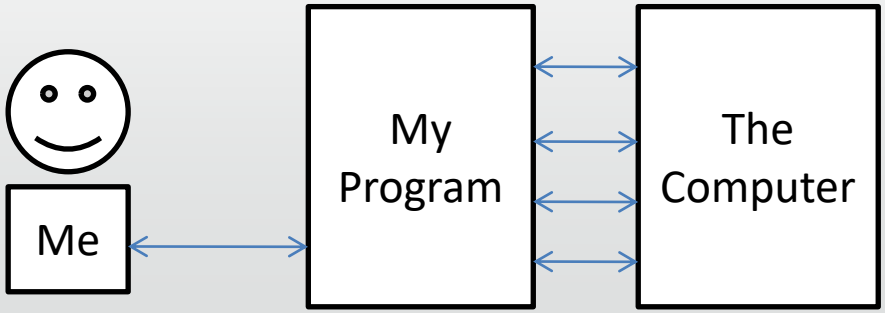
- You write code...



- You **compile** code

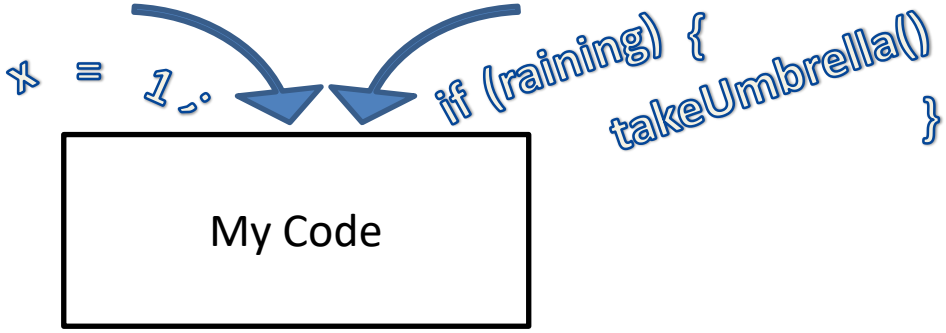


- You run the program

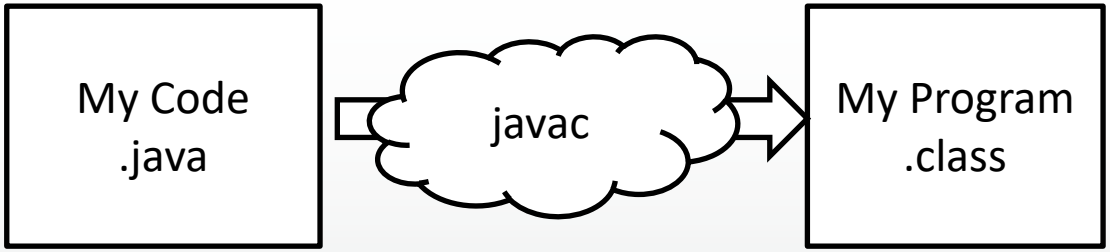


From Java code to program

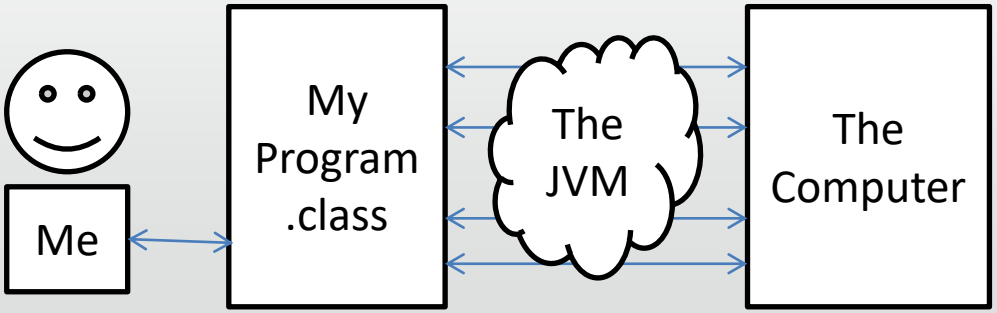
- You write code...



- You **compile** code



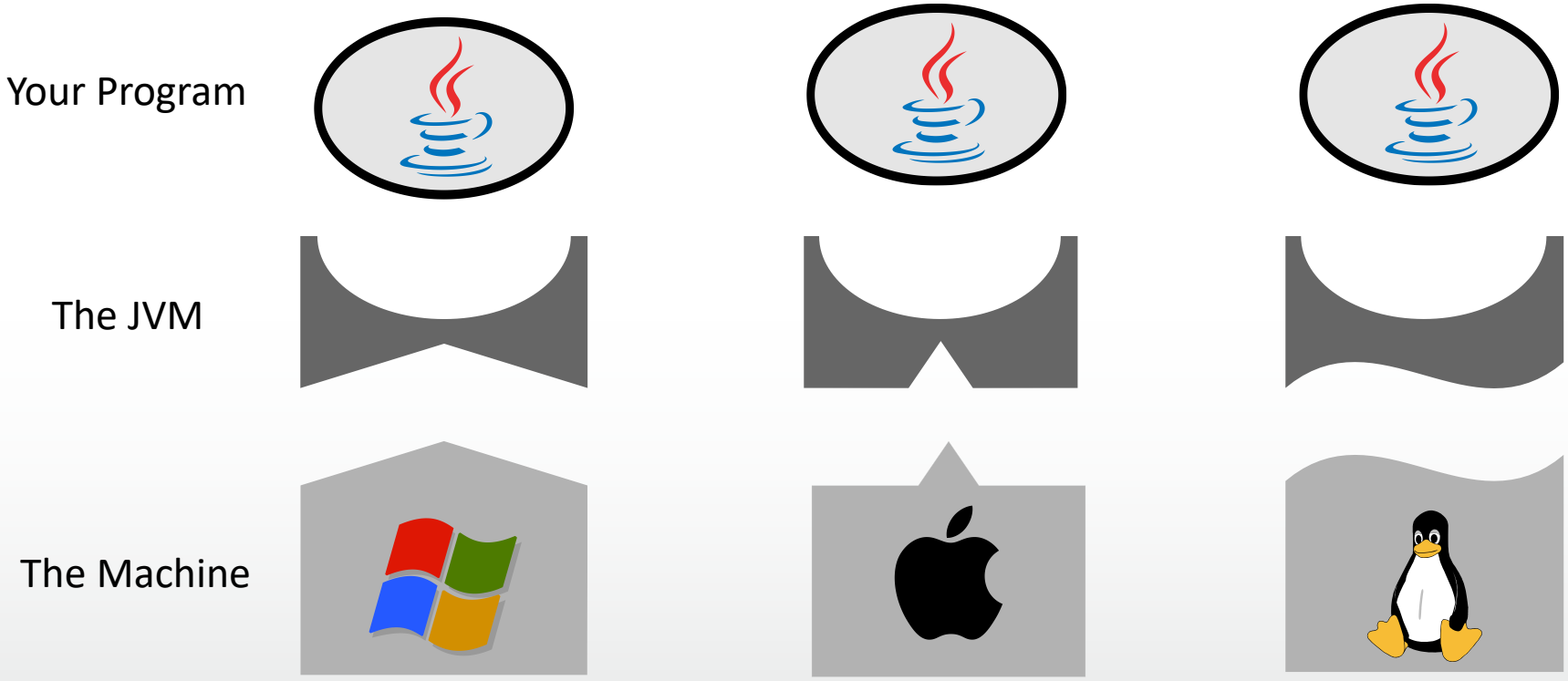
- You run the program



What is a JVM?

- Java Virtual Machine
- Each operating system / machine is different at a very low level:
  - Different way of putting things on the screen
  - Different way of making sound
  - Different way of taking input from keyboard
  - etc…
- So how can Java work on all these platforms?

What is a JVM?



So the JVM is software that allows your Java program to run on any platform where there is a JVM

To compile and run a class

- Save with same name as class
  - e.g., Dog.java
  - (Rule: Must do this)

- Command line to folder

- Compile (create the .class file)
  - `javac Dog.java`

- Run (execute the .class file)
  - `java Dog`

Writing a Class in Java

Structure

Dog.java

```
public class Dog
{
   public String noise = “Woof”;

   public void bark()
   {
         System.out.println(noise);
   }
}
```

Structure

Source file

Dog.java

```
public class Dog
{
    public String noise = “Woof”;

    public void bark()
    {
        System.out.println(noise);
    }
}
```

A Source file is a basic text file. Normally these end .txt, but for java we always save them as .java

Structure

Remember. A class is the blueprint for an object

Source file

Dog.java

Class

```
public class Dog
{
    public String noise = “Woof”;

    public void bark()
    {
        System.out.println(noise);
    }
}
```

Structure

Member variables are how Java handles properties, they store data about the class

It means that all objects (of this class) will contain their own copy of the variable

Source file

Class

Member Variable

Dog.java

```
public class Dog
{
    public String noise = "Woof";

    public void bark()
    {
        System.out.println(noise);
    }
}
```

Structure

Source file

Methods do things, they define behaviour.

Dog.java

Class

Member Variable

Method

```
public class Dog
{
    public String noise = "Woof";

    public void bark()
    {
        System.out.println(noise);
    }
}
```

Structure

Statements are the lines that actually do the operations. Each statement ends in a ;

Source file

Dog.java

Class

Member Variable

Method

Statement

```
public class Dog
{
    public String noise = "Woof";

    public void bark()
    {
        System.out.println(noise);
    }
}
```

Magic Incantations

Magic Incantations

Programming is a complex study

You start learning something…
and find out you need to know
something else to understand it

and something else to understand that

and something else to understand that

Magic Incantations

Dog.java

```
public class Dog
{
    public String noise = "Woof";

    public void bark()
    {
        System.out.println(noise);
    }
}
```

Magic Incantations

You can start going nowhere fast if you try and understand **everything at once**!

We will explain all these things

Go with the flow

You may not understand the ‘incantations’
But don’t let it bother you, trust that they work

You will understand later

Magic Incantation #1

- Where does a program start?
- The program is made up of lots of classes, and those classes have methods, but which method is called **first?**
- In a special method called ‘main’

```
public static void main(String[] args){

}
```

The main method

- Creates the objects you need and tells them what to do, a bit like a conductor in an orchestra
- It doesn't matter which class you put the main method in. You can put it in its own one if you like
  - But there must be 1 and only 1 in the whole program (program is a set of classes that work together)

Dog.java

```
public class Dog
{
    public String noise = "Woof";

    public void bark() {
        System.out.println(noise);
    }

    public static void main(String[] args){
        Dog d = new Dog();
        d.bark();
    }
}
```

Remember

- Your class is just a template
- To make your program work you need objects
  - The instances of your class

```
public static void main(String[] args){
        Dog d = new Dog();
        d.bark();
}
```

A First Example

Banking Example

int and **boolean** are variable **types**, they tell Java what sort of thing is stored in the variables

Note that we can use **//** to show that everything that follows on that line is a **comment** and should be ignored by Java

```
public class Account{

   int balance = 10;            //the bank balance
   boolean active = false;      //true if the account is active

   public void withdrawFiver(){
        balance = balance – 5;
   }
}
```

Making decisions

- Often we want to make the result of a program conditional on something

- If the bank account is not active
  - Don't allow a withdrawal
  - Else, if the account doesn't have enough money
    - Don't allow a withdrawal
  - Else
    - Allow the withdrawal

- For this we need a control structure called **if/else**

Withdrawal

```
public class Account{

    int balance;                //the bank balance
    boolean active;             // true if the account is active
    active = true;              //set active to true

    // some code omitted

    public void withdrawFiver(){
        if (active != true)
            System.out.println("Your account isn't active");

    }
}
```

a != b is a **conditional operator**, it performs a logical test on a and b and returns true if they are not equal.

Other conditionals include <, > and ==

The one statement immediately following the if will only be run if the condition between the () brackets is true

Withdrawal

```
public class Account{

    int balance;                //the bank balance
    boolean active;             // true if the account is active
    active = true;              //set active to true

    //some code omitted

    public void withdrawFiver(){
        if(active != true) {
            System.out.println("Your account isn't active");
            System.out.println("No withdrawal allowed");
        }
    }
}
```

We can use { } to group several statements together so that the if applies to all of them together

Withdrawal

```
public class Account{

    int balance;                //the bank balance
    boolean active;             // true if the account is active
    active = true;              //set active to true

    //some code omitted

    public void withdrawFiver(){
        if(!active){
            System.out.println("Your account isn't active");
            System.out.println("No withdrawal allowed");
        }
    }
}
```

We can simplify

! Is a **logical operator**, it reverses a logical value (so !a returns true if a is false)

Other logical operators are OR (a || b), AND (a && b)

Withdrawal

```
public class Account{

    int balance;                    //the bank balance
    boolean active;                 // true if the account is active
    active = true;                  //set active to true

    //some code omitted

    public void withdrawFiver{
        if (!active) {
            System.out.println("Your account isn't active");
            System.out.println("No withdrawal allowed");
        } else {
            balance = balance - 5;
        }
    }
}
```

Optionally we can add an **else** clause.

Withdrawal

```
public class Account{

     int balance;                          //the bank balance
     boolean active;                       // true if the account is active
     active = true;                        //set active to true

     //some code omitted

     public void withdrawFiver{
          if(!active) {
                    System.out.println("Your account isn't active");
                    System.out.println("No withdrawal allowed");
          } else {
                    if(balance < 5) {
                              System.out.println("Not enough money!");
                    }
                    else {
                              balance = balance - 5;
                    }
          }
     }
}
```

A whole if/else block is actually a single statement, so we can chain them together like this

Withdrawal

```
public class Account{

    int balance;                    //the bank balance
    boolean active;                 // true if the account is active
    active = true;                  //set active to true

    //some code omitted

    public void withdrawFiver(){
        if (!active) {
            System.out.println("Your account isn't active");
            System.out.println("No withdrawal allowed");
        }else if (balance < 5) {
            System.out.println("Not enough money!");
        }
        else {
            balance = balance - 5;
        }
    }
}
```

We can simplify

Because an if/else block is actually considered a single statement we can get rid of some of these brackets and tidy up

The Lab

COMP1202/Labs/Hello World and Conditionals

< COMP1202 | Labs
(Redirected from COMP1202/Labs/1)

Timetable | Labs | Ground Controllers | Space Cadets | Coursework | Exam | Resources] | Back to COMP2012

Lab 1 - Lab 2 - Lab 3 - Lab 4 - Lab 5 - Lab 6 - Lab 7 - Lab 8 - Lab 9 - Revision

navigation
- Main Page
- Recent changes
- Random page
- Help

search
Go Search

toolbox
- What links here
- Related changes
- Upload file
- Special pages
- Printable version
- Permanent link

Contents [hide]

1 Hello World
- 1.1 Introduction
  - 1.1.1 Preparation for this lab
- 1.2 PART 1 - Compiling a class
  - 1.2.1 Compiling and Running
  - 1.2.2 Comments in your code
  - 1.2.3 Things to know
- 1.3 PART 2 - Variables and Object Comparison
  - 1.3.1 A random element
- 1.4 PART 3 - Object Comparison and Program Flow

Hello World [edit]

Introduction [edit]

Welcome to your first COMP1202 lab. Hopefully it will be challenging and fun, and you will enjoy it! If you have any problems remember to ask other members of your lab group and the lab demonstrators for help.

You can discuss the lab here

Make sure you have set up your software. You will need a simple text editor, the Java Development Kit and a command line interface open.

Preparation for this lab [edit]

Have you read over Lecture 2? It contains **important** information that you may need in this lab. Look over it now if you have not already done so

PART 1 - Compiling a class [edit]

[Shortcut: For those who have already studied java, write a Hello World program]

There is a tradition when learning a programming language, that the first program you write should simply output Hello World!

Marking – Functional Correctness

- A+ **Excellent** - Your code compiles and passes all of the test cases
- A **Very Good** - Your code compiles and passes almost all of the test cases
- B **Good** - Your code compiles and passes the majority of the test cases
- C **Acceptable** - Your code compiles and passes some of the text cases
- D **Poor** - Your code does not compile or very few test cases were passed
- E **Inadequate** - No code submitted, not a serious attempt

Marking – Functional Correctness

- A+ **Excellent** - Your code compiles
- A **Very Good** - Your code compiles
- B **Good** - Your code compiles and p
- C **Acceptable** - Your code compiles
- D **Poor** - Your code does not comp
- E **Inadequate** - No code submitted

Automatically Marked

Using Test Harnesses
That we **give to you**
Lab 1 will show you the process

(remember no formal marking until week 4)

Marking – Readability/Coding Style

- A+ **Excellent** - Code is very easy to read and understand, excellent coding style.
  - i.e. Perfect indentation, placement of brackets and camelCase, comments describing design decisions parameters class and method functions.
- A **Very Good** - Minor flaws in style, with an appropriate number of comments but lacking some details
- B **Good** - Code is mostly easy to understand with good programming style with some improvements possible
- C **Acceptable** - Sound programming style but readability needs to be improved
- D **Poor** - Coding style and readability needs significant improvement
- E **Inadequate** - Expected coding style is not used, code is difficult to read
  - i.e. Incorrect indentations, inconsistent placing of brackets, camelCase not used appropriately and no comments

Marking – Readability/Coding Style

- A+ **Excellent** - Code is very easy to rea
  - i.e. Perfect indentation, placement of
    design decisions parameters class and
- A **Very Good** - Minor flaws in style, wit
  lacking some details
- B **Good** - Code is mostly easy to under
  some improvements possible
- C **Acceptable** - Sound programming st
- D **Poor** - Coding style and readability n
- E **Inadequate** - Expected coding style i
  - i.e. Incorrect indentations, inconsisten
    appropriately and no comments

Marked By Demonstrators

Ask for their feedback in the lab sessions.

We will also discuss coding style in week 3.

(remember no formal marking until week 4)

Toolbox

- Getting input and output can be a pain in Java.
- So can some other things

- ECS have provided a ‘toolbox’ for you to use.

- This toolbox is a class of useful methods

- Like readStringfromCmd()

Getting input

```
String theword;  //or whatever name you choose
theword = Toolbox.readStringFromCmd();
```

- Entering this in you code means when the program gets to that line
  - The command prompt will ask you for a string
  - You enter the string and press enter
  - The program keeps going, with the string you typed stored in the variable

Toolbox

- Has some other useful functions, we'll introduce them as we need them
- But you don't have to use them if you know what to do already.

Summary

- How Java Works: The JVM
- Writing a Class in Java
  - Class
  - Member Variables
  - Method
  - Statement
- Magic incantations
  - The main() method
- A First Example
  - Defining an Account class
  - If and Boolean operations
- Introducing Toolbox

Good luck
for the Lab!